Formal Verification: Authority Meet-Semilattice

Zach Kelling, Lux Industries

December 2025

Abstract

We formalize authority as a set of capabilities with meet (intersection). Authority can only narrow through delegation — never escalate. The meet operation forms a semilattice: it is a greatest lower bound that is commutative, associative, and idempotent.

1 Introduction

The authority semilattice ensures that delegation in the Lux trust model is monotonically narrowing. No delegation can grant more authority than the delegator possesses. This is the algebraic foundation for role-based access control in Hanzo IAM.

2 Definitions

Definition 1 (Capability)**.** *Capabilities: shell, exec, build, attest, sign, deploy.*

Definition 2 (Authority)**.** *A set of capabilities.*

3 Theorems and Axioms

Theorem 3 (`meet_le_left`)**.** *Meet is a lower bound (left):* $a \sqcap b \leq a$.

Proof. Filter preserves membership in a. □

Theorem 4 (`meet_le_right`)**.** *Meet is a lower bound (right):* $a \sqcap b \leq b$.

Proof. Filter checks membership in b via `contains`. □

Theorem 5 (`meet_glb`)**.** *Meet is the greatest lower bound: if* $c \leq a$ *and* $c \leq b$*, then* $c \leq a \sqcap b$.

Theorem 6 (`none_is_bottom`)**.** *Empty authority is the bottom element:* $\bot \leq a$ *for all* a.

Theorem 7 (`delegation_narrows`)**.** *Delegating authority can only reduce it:* $delegator \sqcap scope \leq delegator$.

4 Lean Source

```
/-
  Authority Meet-Semilattice

  Models authority as a set of capabilities with meet (intersection).
  Authority can only NARROW through delegation - never escalate.

  Maps to:
  - hanzo/iam: RBAC permission sets
  - lux/node: validator capability scoping
  - zoo/contracts: role-based access control

  Properties:
  - Meet is commutative, associative, idempotent
  - Authority narrows through delegation (monotone)
  - Meet is greatest lower bound
-/

import Mathlib.Data.Nat.Defs
import Mathlib.Data.List.Basic
import Mathlib.Tactic

namespace Trust.Authority

/-- A capability: what you're allowed to do -/
inductive Capability where
  | shell (user : String)
  | exec (command : String)
  | build (target : String)
  | attest (scope : String)
  | sign (keyId : String)
  | deploy (env : String)
  deriving DecidableEq, Repr

/-- Authority: a set of capabilities (List for simplicity) -/
structure Authority where
  capabilities : List Capability
  deriving DecidableEq, Repr

/-- No authority -/
def Authority.none : Authority :=  []

/-- Full authority -/
def Authority.all : Authority := [
  .shell "root", .exec "*", .build "*",
  .attest "*", .sign "*", .deploy "*"
]

/-- Meet (intersection): only capabilities in BOTH sets -/
def Authority.meet (a b : Authority) : Authority :=
  a.capabilities.filter (b.capabilities.contains ·)

/-- Subset ordering -/
```

```
def Authority.subset (a b : Authority) : Prop :=
   c   a.capabilities, c   b.capabilities

instance : LE Authority where le := Authority.subset
instance : Min Authority where min := Authority.meet

/-- Meet is a lower bound (left): a   b   a -/
theorem meet_le_left (a b : Authority) : min a b   a := by
  intro c hc
  simp [Min.min, Authority.meet, List.mem_filter] at hc
  exact hc.1

/-- Meet is a lower bound (right): a   b   b -/
theorem meet_le_right (a b : Authority) : min a b   b := by
  intro c hc
  simp [Min.min, Authority.meet, List.mem_filter, List.contains_eq_any_beq,
        List.any_eq_true] at hc
  obtain  _,  x, hxb,  hcx := hc
  simp [beq_iff_eq] at hcx; rw [hcx]; exact hxb

/-- Meet is greatest lower bound -/
theorem meet_glb (a b c : Authority) (hca : c   a) (hcb : c   b) :
    c   min a b := by
  intro cap hcap
  simp [Min.min, Authority.meet, List.mem_filter, List.contains_eq_any_beq,
        List.any_eq_true]
  exact  hca cap hcap, cap, hcb cap hcap, beq_self_eq_true  cap

/-- Authority.none is the bottom element -/
theorem none_is_bottom (a : Authority) : Authority.none   a := by
  intro c hc; simp [Authority.none] at hc

/-- DELEGATION NARROWING: delegating authority can only reduce it -/
theorem delegation_narrows (delegator_auth delegated_scope : Authority) :
    min delegator_auth delegated_scope   delegator_auth :=
  meet_le_left delegator_auth delegated_scope

end Trust.Authority
```

5 References

Source code: https://github.com/luxfi/proofs/blob/main/lean/Trust/Authority.lean
White papers: https://papers.lux.network
Related white papers:

- lux-id-iam.tex